Listas Encadeadas

Prof. Ricardo Mesquita

Em Listas Sequenciais...

Conjunto de itens organizados → vetor

- a organização é implícita (pela posição)

- o símbolo **vet** representa o endereço do primeiro elemento (ponteiro)
- ocupa um espaço contíguo na memória:
  - permite acesso a qualquer elemento a partir do ponteiro para o primeiro, utilizando indexação → acesso aleatório

Em Listas Sequenciais...

Qual a principal desvantagem de se usar o armazenamento sequencial para representar Listas?

→ **Quantidade fixa de elementos**

- **memória alocada sem uso ou**
- **impossibilidade de alocar mais memória**

Em Listas Sequenciais...

Solução?

- Utilizar Estruturas de Dados que cresçam e diminuam na medida da necessidade → Estruturas Dinâmicas
- Alocação dinâmica de memória para armazenar os elementos

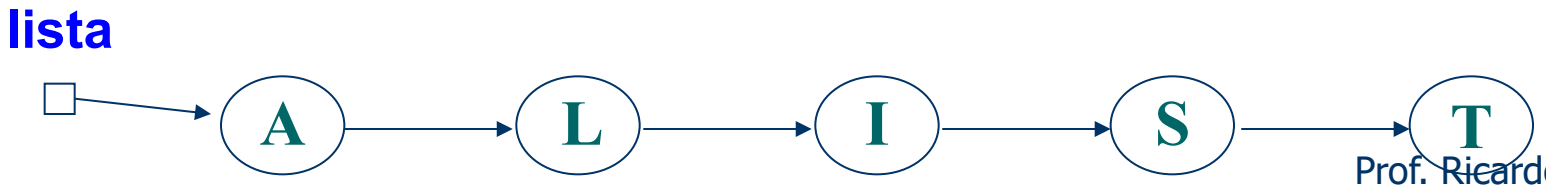
Listas Encadeadas

Listas Encadeadas

- Podem crescer e diminuir dinamicamente
- Tamanho máximo não precisa ser definido previamente
- Proveem flexibilidade, permitindo que os itens sejam rearrumados eficientemente
  - perda no tempo de acesso a qualquer item arbitrário da lista, comparando com vetores
- Também chamadas de Listas Ligadas

Listas Encadeadas

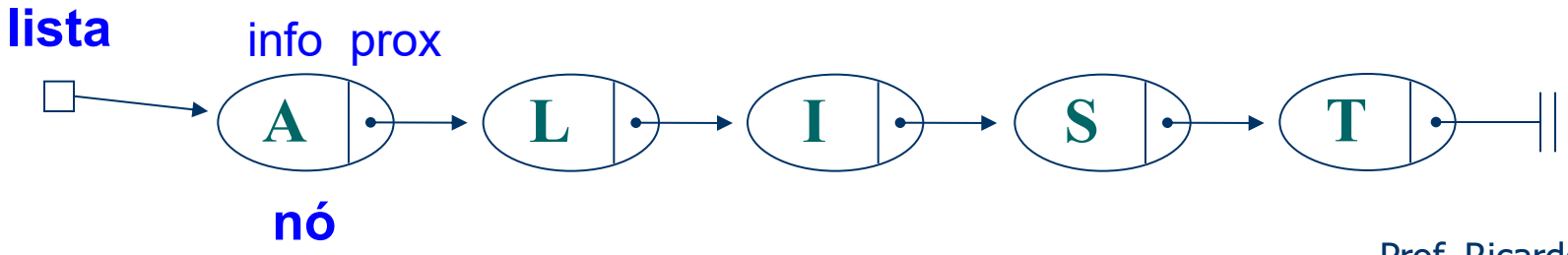
- A sequência de elementos é especificada explicitamente, onde cada um contém um ponteiro para o próximo da lista (*link*) → **Encadeamento**
- Cada elemento é chamado de **nó da lista**
- A lista é representada por um ponteiro para o primeiro elemento (ou nó)
- Do primeiro elemento, pode-se alcançar o segundo seguindo o encadeamento e assim sucessivamente
- Para cada novo elemento inserido na estrutura, um espaço na memória é alocado dinamicamente, mas a alocação do espaço não é contígua



Listas Encadeadas

Detalhes que devem ser considerados:

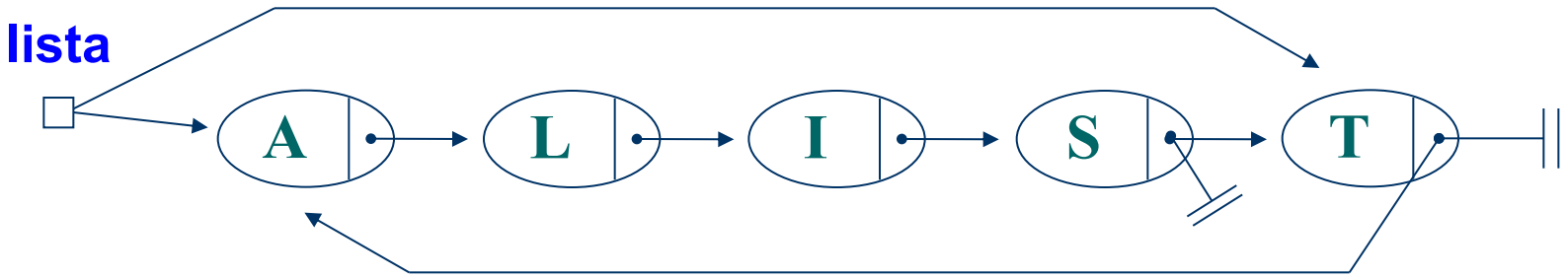
- cada elemento possui pelo menos dois campos: um para armazenar a **informação** e outro para o endereço do **próximo** (ponteiro)
- deve haver um ponteiro especial para o 1º da lista
- o ponteiro do último elemento tem que especificar algum tipo de *final* (aponta para si próprio ou **nulo**)
- uma lista vazia (ou nula) é uma lista sem nós



Listas Encadeadas

Algumas operações são mais eficientes do que em Lista Sequencial

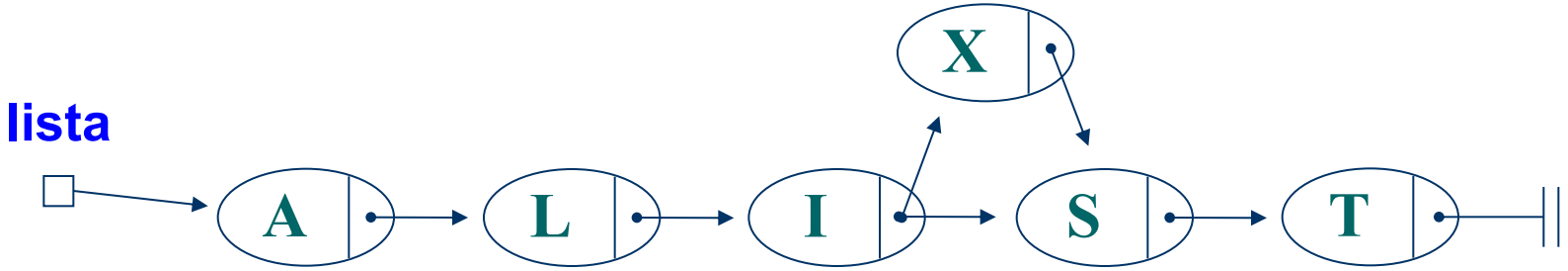
- Mover o elemento com a informação **T** do fim da lista para o início
- Mesmo que a lista seja muito longa, a mudança estrutural é realizada através de 3 operações



Listas Encadeadas

Para inserção de um novo elemento X:

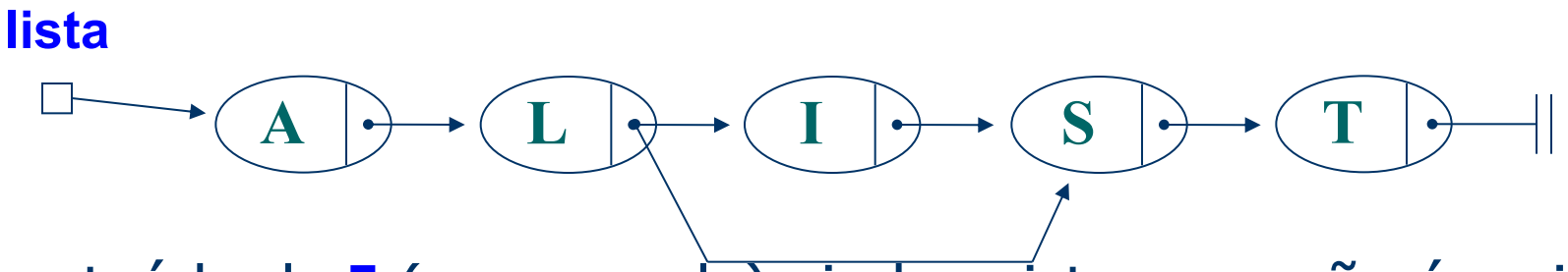
- aloca-se memória para um elemento e atualiza-se os ponteiros
- em lista sequencial seria necessário deslocar todos os elementos a partir do ponto de inserção;
- apenas 2 *links* são alterados para esta operação - não importa quão longa é a lista



Listas Encadeadas

Remoção de um elemento:

- Basta alterar o ponteiro do elemento anterior ao removido



- o conteúdo de **I** (no exemplo) ainda existe, mas não é mais acessível pela lista

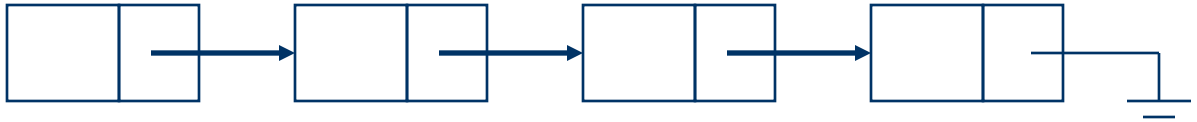
Listas Encadeadas

Lista Encadeada × Sequencial

- remoção e inserção são mais naturais na lista encadeada
- outras operações já não são tão naturais
  - Encontrar o k-ésimo elemento da lista
    - em uma lista sequencial - simplesmente acessar a[k-1]
    - na lista encadeada é necessário percorrer k *links*
  - Achar um item localizado antes de um outro item

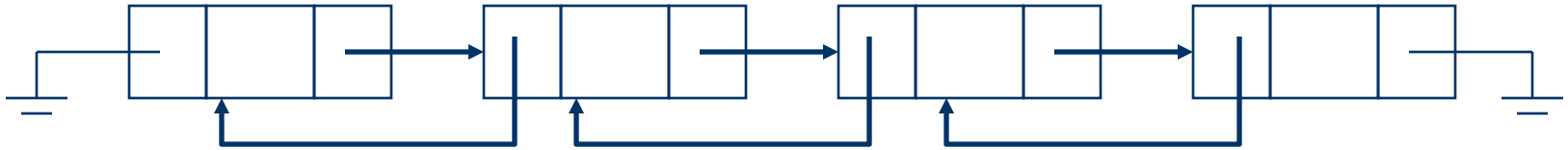
Listas Encadeadas

Simplesmente encadeada - ponteiros em uma direção



Duplamente encadeada - ponteiros para duas direções

- um ponteiro para o próximo e um para o anterior



Listas Encadeadas

Implementações:

```
typedef int tpitem;

typedef struct tp_no {
    tpitem info;
    struct tp_no *prox;
} tplista;
tplista *lista;
```

Listas Encadeadas

- Inicialização da Lista:

```
lista=NULL;
```

- Lista Vazia:

```
int vazia (tplista *t) {
    return (t==NULL);
}
```

Listas Encadeadas

- Alocação Dinâmica de Memória
  - **malloc** ➔ aloca( );
  - **free**
  - exigem #include "stdlib.h"

Manipulação da Memória

Função **malloc**

- aloca dinamicamente uma parte da memória
- recebe como parâmetro o número de bytes que se deseja alocar
- retorna um ponteiro para endereço inicial da área alocada
- se não houver espaço livre, retorna um endereço nulo (representado por NULL, definido em *stdlib.h*) que pode ser testado para encerrar o programa

Manipulação da Memória

- Função **malloc**
  - Ex.:

```
int *p;
p = malloc(8);
```

  - p representa o endereço inicial de uma área contínua de memória suficiente para armazenar 8 bytes

Manipulação da Memória

- Função **malloc**
  - utilização da função sizeof( )

```
int *p;
p = malloc(sizeof(int));
```

  - ponteiro retornado é para um item do tipo *char*

➔ converter o tipo (*cast*)

```
int *p;
p = (int *) malloc(sizeof(int));
```

Manipulação da Memória

aloca()

```
tplista* aloca( ) {
    tplista* pt;
    pt=(tplista*) malloc(sizeof(tplista));
    return pt;
}
```

No bloco principal:

```
        p=aloca( );
        if (p!=NULL)
                /* continua o programa... */
        else
                printf("Não há memória disponível!");
```

Manipulação da Memória

- Função **free**
  - Libera o espaço de memória alocado dinamicamente
  - recebe como parâmetro o ponteiro da área de memória a ser liberada
  - Ex.: `free(p);`

Listas Encadeadas

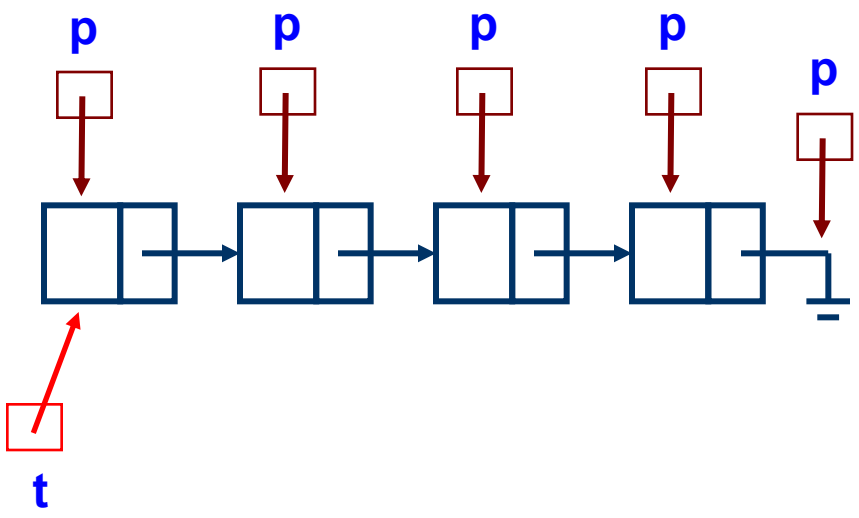
- Operações Básicas
  - vazia(lista);
  - insere(&lista, valor);
  - busca(lista, valor);
  - listagem(lista);
  - retira(&lista, valor);

Lista Vazia

```
int vazia (tplista *t) {
        if (t==NULL)
                return 1;
        else
                return 0;
}
```

Listagem

- Utilizar um ponteiro auxiliar para percorrer a lista até o final



Listagem

```
void listagem (tplista *t) {
    tplista *p;
    for (p=t; p!=NULL; p=p->prox)
        printf("Info: %d\n", p->info);
}
```

Busca

- Utilizar um ponteiro auxiliar para percorrer a lista até encontrar ou até chegar ao final
- A função retorna o ponteiro para o elemento ou *NULL* caso não encontre

Busca

```
tplista* busca (tplista *t , tpitem valor) {
   tplista *p=t;
    while ((p!=NULL) && (p->info!=valor))
        p=p->prox;
   return p;
}
```

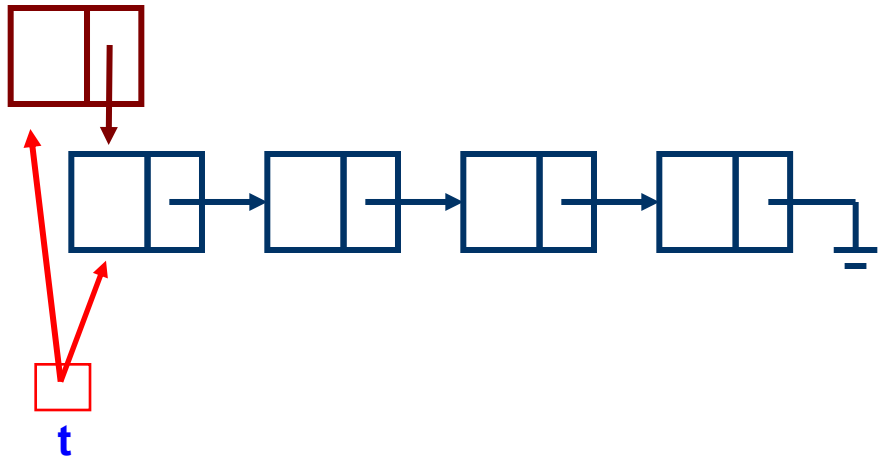
Inserção no Início

- Cada elemento armazena uma informação do tipo *tp_item*
- A função:
  - recebe como parâmetros um ponteiro para a lista e o novo elemento
  - tenta alocar dinamicamente o espaço para o novo nó, guardando a informação e fazendo ele apontar para o início da lista
  - retorna o novo ponteiro da lista ou NULL caso não seja possível incluir

Inserção no Início

Estrutura usada:

```
typedef int tpitem;

typedef struct tp_no {
    tpitem info;
    struct tp_no *prox;
} tplista;
tplista * lista;
```



Inserção no Início

```
tplista * insere (tplista *t , tpitem valor) {
    tplista *novo;
    novo = aloca( );
    if (!novo)
        return(NULL);
    else {
        novo->info = valor;
        novo->prox = t;
        return(novo);
    }
}
```

Insere em qualquer lista, retornando um ponteiro para o primeiro. Quem chama a função deve atualizar o ponteiro da lista com o valor retornado:

```
tmp= insere(lista,12);
if (tmp)
    lista=tmp;
else
    printf("Sem memória");
```

Inserção no Início (Por referência)

```
int  insere (tplista **t , tpitem valor) {
    tplista *novo;
    novo = aloca( );
    if (!novo)
        return 0;
    else {
        novo->info = valor;
        novo->prox = *t;
        *t=novo;
        return 1;
    }
}
```

Insere em qualquer lista, retornando 1 ou zero para indicar sucesso ou falta de memória. A função já atualiza automaticamente o ponteiro de início da lista.

```
ok= insere(&lista,12);
if (!ok)
    printf("Sem memória");
```

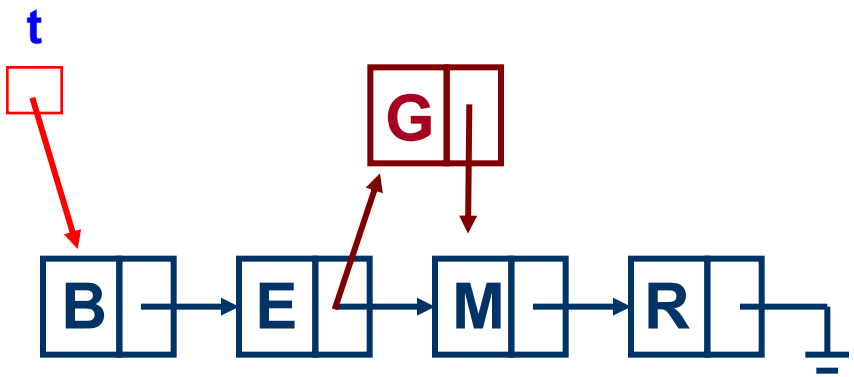
Inserção Geral

- Insere elementos em uma lista classificada, mantendo a ordenação
- Ponto chave: localizar o elemento da lista que precede o novo
- De posse do ponteiro para este antecessor, pode-se encadear o novo elemento:
  - o novo apontará para o próximo do antecessor
  - o antecessor apontará para o novo

Inserção Geral

Estrutura usada:

```
typedef int tpitem;

typedef struct tp_no {
    tpitem info;
    struct tp_no *prox;
} tplista;
tplista * lista;
```



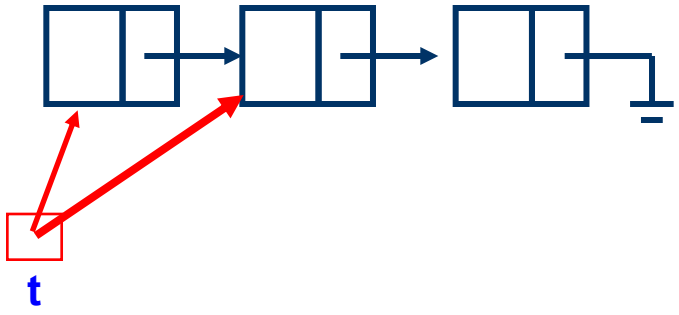
tplista* insere (tplista *t , tpitem e) {

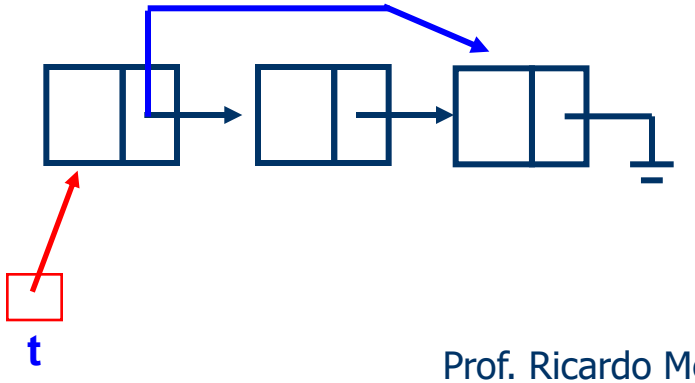
```
/* novo nó */
tplista *novo;

/* ponteiro p/ anterior */
tplista *ant=NULL;

/* ponteiro p/ percorrer */
tplista *p=t;

novo = aloca();
if (!novo)
      return(NULL);
novo->info = e;

/* procura posição do elemento */
while (p!=NULL && p->info < e) {
      ant=p;
      p=p->prox;  }

if (ant==NULL) {
/* insere elemento no início */
      novo->prox=t;
      t=novo; }
else {
/* insere elemento no meio */
      novo->prox=ant->prox;
      ant->prox=novo; }
return t;
}
```

```
int insere (tplista **t , tpitem e) {
    /* Por Referência */
    tplista *novo;
    tplista *ant=NULL;
    tplista *p=*t;
    novo = aloca();
    if (!novo)
        return 0;
    novo->info = e;
    while (p && p->info < e) {
        ant=p;
        p=p->prox;
    }
    novo->prox=p;
    if (ant==NULL)
        *t=novo;
    else
        ant->prox=novo;
    return 1;
}
```

Remoção

- Procurar o elemento a ser removido e guardar uma referência para o anterior
- Se encontrar:
  - se for o primeiro: lista passa a apontar para o segundo e espaço do primeiro é liberado
  - se estiver no meio ou fim: seu antecessor passa a apontar para seu próximo e o espaço é liberado





tp_lista* retira (tp_lista *t, tpitem e) {

```
/* ponteiro p/ anterior */
tp_lista *ant=NULL;

/* ponteiro p/ percorrer */
tp_lista *p=t;
/* procura elemento na lista,
guardando o anterior */
while (p!=NULL && p->info != e)
{
        ant=p;
        p=p->prox;
}
```

```
    if (p==NULL) {
    /* Não Achou; retorna NULL  */
            printf("Não existe\n");
            return NULL; }
/* Achou */
    if (ant==NULL) {
    /* retira elemento do início */
            t=p->prox; }
    else {
    /* retira elemento do meio */
            ant->prox=p->prox;    }
    free(p);
    return t;
}
```

```
int retira (tplista **t, tpitem e) {
    /* Por Referência */
    tplista *ant=NULL, *p=*t;
    while (p!=NULL && p->info!=*e)
    {
        ant=p;
        p=p->prox;
    }
    if (p==NULL)
        return 0;
    else {
        if (ant==NULL)
            *t=p->prox;
        else
            ant->prox=p->prox;
        free(p);
        return 1;
    }
}
```

Liberar Lista

- Liberar todos os elementos alocados
  - ➔ Percorrer todos os elementos, liberando cada um
- É preciso guardar a referência para o próximo antes de liberar o atual

Liberar Lista

```
void destroi (tplista *t) {
    tplista *p=t, *q;
    while (p!=NULL) {
        q=p->prox; /* guarda referência p/ o próximo */
        free(p); /* libera a memória apontada por p */
        p=q; /* faz p apontar para o próximo */
    }
}
```

Praticar!